

Versión 0.1.2

Proyecto de Machine Learning/Aprendizaje Automático - Parte 1: Visualización de Datos mediante Motor de Charting usando la librería Tk

Desarrollo de un motor de gráficos financieros con indicadores técnicos mediante la librería Tk.

1. Objetivo:

El objetivo de la primera parte de este proyecto es desarrollar un sistema funcional equivalente a una plataforma de gráficos financieros como TradingView (www.tradingview.com), el cual permitirá trabajar el componente de Visualización de Datos de Machine Learning/Aprendizaje Automático, el cual permitirá la oportuna integración de los indicadores y visualización de datos necesarios para entrenar modelos predictivos recurrentes a lo largo del segundo bimestre.

2. Conceptos clave que aprenderán

Este proyecto cubre:

Matemática

- Transformaciones lineales
- Escalado de datos

Machine Learning/Aprendizaje Automático

- Visualización de Datos
- Desarrollo de indicadores
- Extracción de características
- Análisis de datos (2do bimestre)
- Desarrollo de modelos predictivos recurrentes (2do bimestre)

Programación

- Arquitectura modular
- Programación Orientada a Objetos
- Separación de responsabilidades (Técnicas de Ingeniería de Software)

Trading systems

- OHLC
 - Indicadores técnicos
 - Visualización financiera
-

3. Arquitectura del proyecto

El sistema está dividido en 4 capas:

1. Datos

- `Market/MarketData.pm`
- Maneja etiquetas temporales, velas, precios, volumen

2. Indicadores

- `Market/IndicatorManager.pm`
- `Market/Indicators/ATR.pm`
- Calculan señales (ATR, etc.)

3. Renderizado

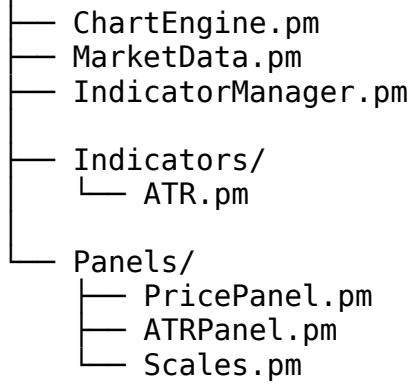
- `Market/ChartEngine.pm`
- `Market/Panels/PricePanel.pm`
- `Market/Panels/ATRPanels.pm`
- `Market/Panels/Scales.pm`

4. Aplicación

- `market.pl`
-

4. Estructura

Market/



5. Funcionalidades que deben replicar desde TradingView

Básico

- Velas
- Eje vertical de precios
- Eje vertical del indicador ATR
- Eje horizontal de tiempo.
- Scroll horizontal (arrastre de mouse)

Intermedio

- Zoom vertical automático / vertical manual (barra de precios) / zoom horizontal (eje temporal)
- Crosshair sincronizado entre todos los paneles
- Múltiples panel sincronizados en el tiempo

Avanzado

- Indicadores desacoplados
 - Escalas verticales independientes por panel
 - Render incremental – Complejidad $O(1)$
 - Temporalidades de 1, 5 y 15 minutos
-

6. Metodología de trabajo

- Cada método debe implementarse respetando estrictamente:
- Inputs definidos
- Outputs esperados
- Responsabilidad única de cada módulo
- Documentación del código
- Buenas prácticas de codificación
- Técnicas de Ingeniería de Software
- Validar comportamiento que sea idéntico al de TradingView

⚠ Prohibido:

- No mezclar la lógica de cálculo con renderizado
 - No usar variables globales
 - No acoplar indicadores al chart
 - Utilizar otras librerías sino las que están definidas en el código dado
-

7. Guía de implementación por módulos

Instalación de librerías:

```
sudo cpanm Time::Moment
```

```
sudo dnf install perl-Tk
```

```
sudo cpanm Tk
```

```
sudo cpanm Tk::Chart
```

```
sudo cpanm Chart::Clicker
```

Definición de las Plantillas de Trabajo:

Las plantillas descritas a continuación definen la estructura completa de un sistema equivalente a una plataforma de gráficos financieros como TradingView. Cada grupo de estudiantes deberá completar la lógica siguiendo las descripciones.

Market/MarketData.pm

Descripción

Clase responsable de almacenar y gestionar los datos de mercado (OHLCV). Debe garantizar sincronización temporal, acceso eficiente por índice y actualización incremental de datos.

new

Inicializa almacenamiento de datos OHLC.

get_data

Devuelve la estructura completa de datos.
Acceso general.

add_candle

Agrega una vela nueva.
Entrada principal de datos.

build_tf_candles

Construye velas en una temporalidad específica.
Agregación (ej: 1m → 5m).

build_timeframes

Construye todas las temporalidades disponibles.
Preprocesamiento completo.

set_timeframe

Selecciona la temporalidad activa.
Afecta qué datos se usan.

`_active_array`

Devuelve el array activo según timeframe.
Abstracción interna clave.

`get_slice`

Devuelve un subconjunto de velas.
Base para indicadores y render.

`get_candle`

Obtiene una vela por índice.

`size`

Número total de velas.

`last_candle`

Devuelve la última vela.

`last_index`

Devuelve índice de la última vela.

`get_timestamp`

Obtiene timestamp de una vela.

`merge_delta_row`

Actualiza o inserta datos incrementales.
Manejo de streaming.

compute_time_anchors

Calcula puntos clave de tiempo.
Usado para ejes o etiquetas.

```
package Market::MarketData;
```

```
use strict;  
use warnings;
```

```
sub new {  
    my ($class) = @_;  
    my $self = {  
        data => {},  
    };  
    bless $self, $class;  
    return $self;  
}
```

```
sub get_data {  
    my ($self) = @_;  
    # TODO  
}
```

```
sub add_candle {  
    my ($self, $candle) = @_;  
    # TODO  
}
```

```
sub build_tf_candles {  
    my ($self, $tf) = @_;  
    # TODO  
}
```

```
sub build_timeframes {  
    my ($self) = @_;  
    # TODO  
}
```

```
sub set_timeframe {  
    my ($self, $tf) = @_;  
    # TODO  
}
```

```
sub _active_array {
```

```

my ($self) = @_;
# TODO
}

sub get_slice {
my ($self, $start, $end) = @_;
# TODO
}

sub get_candle {
my ($self, $index) = @_;
# TODO
}

sub size {
my ($self) = @_;
# TODO
}

sub last_candle {
my ($self) = @_;
# TODO
}

sub last_index {
my ($self) = @_;
# TODO
}

sub get_timestamp {
my ($self, $index) = @_;
# TODO
}

sub merge_delta_row {
my ($self, $row) = @_;
# TODO
}

sub compute_time_anchors {
my ($self) = @_;
# TODO
}

1;

```

Market/IndicatorManager.pm

Descripción

Gestiona múltiples indicadores técnicos de forma desacoplada. Permite registrar, actualizar y consultar indicadores sin acoplarlos al sistema de render.

`new`

Inicializa el contenedor de indicadores.

`register`

Registra un indicador.
Permite extensibilidad.

`update_last`

Actualiza indicadores con la última vela.
Cálculo incremental eficiente.

`get`

Obtiene valores de un indicador.

`slice_array`

Devuelve una porción de valores del indicador.
Sincronización con ventana visible.

`reset_all`

Reinicia todos los indicadores.
Útil al cambiar timeframe.

```
package Market::IndicatorManager;
```

```
use strict;
use warnings;

sub new {
    my ($class) = @_;
    my $self = {
        indicators => {},
    };
    bless $self, $class;
    return $self;
}

sub register {
    my ($self, $name, $indicator) = @_;
    # TODO
}

sub update_last {
    my ($self, $market_data) = @_;
    # TODO
}

sub get {
    my ($self, $name) = @_;
    # TODO
}

sub slice_array {
    my ($self, $name, $start, $end) = @_;
    # TODO
}

sub reset_all {
    my ($self) = @_;
    # TODO
}

1;
```

Market/Indicators/ATR.pm

Descripción

Implementa el indicador Average True Range (ATR). Debe calcular volatilidad basada en precios históricos.

Conceptos que deben investigar:

- Average True Range
- Media móvil (SMA / Wilder)

Validación:

Comparar ATR con TradingView

new

Inicializa ATR con su período.

update_last

Actualiza el ATR con la última vela.
Implementa cálculo incremental.

get_values

Devuelve serie completa del ATR.

reset

Reinicia el indicador.

```
package Market::Indicators::ATR;
```

```
use strict;  
use warnings;
```

```
sub new {  
  my ($class, $period) = @_;  
  my $self = {  
    period => $period,  
    values => [],
```

```
};  
bless $self, $class;  
return $self;  
}  
  
sub update_last {  
my ($self, $market_data) = @_;  
# TODO  
}  
  
sub get_values {  
my ($self) = @_;  
# TODO  
}  
  
sub reset {  
my ($self) = @_;  
# TODO  
}  
  
1;
```

Market/Panels/Scales.pm

Descripción

Cada panel tiene su propio eje vertical Y. Por otro lado, el horizontal X es común entre todos los paneles, el cual sirve para el zoom, el scroll y crosshair cuyas coordenadas horizontales son las mismas entre todos los paneles. La clase también gestiona la transformación entre índices de datos y coordenadas en pantalla de los ejes vertical y horizontal. Importante: NUNCA mezclar coordenadas de datos con coordenadas de pantalla.

new

Inicializa sistema de escalas.

index_to_x

Convierte índice → coordenada X.

x_to_index

Convierte X → índice entero.

x_to_index_float

Convierte X → índice continuo.
Más precisión para interacción.

index_to_center_x

Devuelve centro de una vela en X.

value_to_y

Convierte valor (precio/indicador) → Y.

y_to_value

Convierte Y → valor.

`_draw_y_scale`

Dibuja escala vertical (precios/valores).

```
package Market::Panels::Scales;
```

```
use strict;  
use warnings;
```

```
sub new {  
    my ($class, %args) = @_;  
    my $self = {  
        %args,  
    };  
    bless $self, $class;  
    return $self;  
}
```

```
sub index_to_x {  
    my ($self, $index) = @_;  
    # TODO  
}
```

```
sub x_to_index {  
    my ($self, $x) = @_;  
    # TODO  
}
```

```
sub x_to_index_float {  
    my ($self, $x) = @_;  
    # TODO  
}
```

```
sub index_to_center_x {  
    my ($self, $index) = @_;  
    # TODO  
}
```

```
sub value_to_y {  
    my ($self, $value) = @_;  
    # TODO  
}
```

```
sub y_to_value {  
    my ($self, $y) = @_;  
    # TODO  
}
```

```
sub _draw_y_scale {  
  my ($self, $canvas) = @_;  
  # TODO  
}
```

```
1;
```

Market/Panels/PricePanel.pm

Descripción

Renderiza el gráfico principal de precios (velas). Maneja escalado vertical, dibujo de velas y crosshair.

Tareas:

- Dibujar velas OHLC
- Escalar precios a pixeles
- Dibujar eje de precios
- Mostrar precio conforme la ubicación del cursor de mouse en pantalla

new

Inicializa el panel de precios.

_init_crosshair_objects

Crea elementos gráficos del crosshair.
Preparación visual.

round

Redondeo auxiliar.

render

Dibuja las velas visibles.
Función principal del panel.

render_last_visible_price

Dibuja el último precio visible.
Información destacada.

get_y_range

Calcula min/max de precios visibles.
Base para escalado vertical.

set_scale

Asigna escala de valores a píxeles.

draw_crosshair

Dibuja el crosshair en este panel.

draw_time_axis

Dibuja el eje temporal.
Etiquetas de tiempo.

```
package Market::Panels::PricePanel;

use strict;
use warnings;

sub new {
    my ($class, %args) = @_;
    my $self = {
        %args,
    };
    bless $self, $class;
    return $self;
}

sub _init_crosshair_objects {
    my ($self) = @_;
    # TODO
}

sub round {
    my ($self, $value) = @_;
    # TODO
}
```

```
sub render {  
  my ($self, $canvas, $data, $scale) = @_;  
  # TODO  
}
```

```
sub render_last_visible_price {  
  my ($self, $canvas) = @_;  
  # TODO  
}
```

```
sub get_y_range {  
  my ($self, $data) = @_;  
  # TODO  
}
```

```
sub set_scale {  
  my ($self, $scale) = @_;  
  # TODO  
}
```

```
sub draw_crosshair {  
  my ($self, $x, $y) = @_;  
  # TODO  
}
```

```
sub draw_time_axis {  
  my ($self, $canvas, $timestamps) = @_;  
  # TODO  
}
```

```
1;
```

Market/Panels/ATRPanel.pm

Descripción

Renderiza el indicador ATR en un panel separado con su propia escala.

Tareas:

- Graficar línea ATR
- Crear escala independiente
- Mostrar valor ATR dinámico

new

Inicializa panel ATR.

_init_crosshair

Configura crosshair del panel.

get_y_range

Calcula rango del ATR visible.

set_scale

Define escala vertical.

render

Dibuja línea del ATR.

render_last_visible_value

Muestra último valor ATR.

draw_crosshair

Dibuja crosshair sincronizado.

```
package Market::Panels::ATRPanels;

use strict;
use warnings;

sub new {
    my ($class, %args) = @_;
    my $self = {
        %args,
    };
    bless $self, $class;
    return $self;
}

sub _init_crosshair {
    my ($self) = @_;
    # TODO
}

sub get_y_range {
    my ($self, $values) = @_;
    # TODO
}

sub set_scale {
    my ($self, $scale) = @_;
    # TODO
}

sub render {
    my ($self, $canvas, $values, $scale) = @_;
    # TODO
}

sub render_last_visible_value {
    my ($self, $canvas) = @_;
    # TODO
}

sub draw_crosshair {
    my ($self, $x, $y) = @_;
    # TODO
}

1;
```

Market/ChartEngine.pm

Descripción

Inicializa el motor del gráfico.

- Recibe referencias a market, indicators, canvases y widgets Tk.
- Define estado interno: zoom, offset, crosshair, render flags.
- Instancia los paneles (PricePanel y ATRPanel).

Es el punto de ensamblaje del sistema visual: Orquesta el renderizado completo del sistema. Coordina paneles, escalas y eventos del usuario.

compute_window

Calcula qué porción de datos es visible.

- Usa visibleBars, zoom y offset.
Define el rango de índices a renderizar.

round

Redondeo numérico auxiliar.

Usado para cálculos de píxeles o precios.

request_render

Solicita un render diferido.

- Evita renderizados redundantes.
Optimización clave para rendimiento en Tk.

render

Dibuja todo el gráfico.

- Calcula ventana visible
 - Llama a render de cada panel
Es el **loop de render principal**.
-

`_bind_all_canvas`

Asocia eventos a múltiples canvas.
Permite interacción uniforme.

`bind_events`

Registra eventos de mouse/teclado.
Activa zoom, drag, crosshair.

`_horizontal_zoom`

Controla zoom horizontal (cantidad de velas visibles).
Modifica `visibleBars`.

`_vertical_drag`

Controla desplazamiento vertical manual.
Ajusta rango Y cuando no está en modo automático.

`_vertical_zoom`

Controla zoom vertical.
Escala el eje de precios.

`_on_mouse_move`

Maneja movimiento del mouse.

- Actualiza posición del crosshair
Entrada principal de interacción.

`_draw_crosshair_all`

Dibuja crosshair en todos los paneles.
Sincroniza visualmente los paneles.

set_timeframe

Cambia la temporalidad del mercado.
Requiere reconstrucción de datos.

reset_view

Resetea zoom y desplazamiento.
Vuelve al estado inicial.

compute_intraday_labels

Calcula etiquetas de tiempo para el eje X.
Maneja formato temporal.

get_all_timestamps

Devuelve timestamps visibles.
Usado para ejes o sincronización.

```
package Market::ChartEngine;
```

```
use strict;  
use warnings;
```

```
sub new {  
  my ($class, %args) = @_;  
  my $self = {  
    %args,  
  };  
  bless $self, $class;  
  return $self;  
}
```

```
sub compute_window {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub round {  
  my ($self, $value) = @_;  
  # TODO  
}
```

```
sub request_render {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub render {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub _bind_all_canvas {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub bind_events {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub _horizontal_zoom {  
  my ($self, $delta) = @_;  
  # TODO  
}
```

```
sub _vertical_drag {  
  my ($self, $dy) = @_;  
  # TODO  
}
```

```
sub _vertical_zoom {  
  my ($self, $factor) = @_;  
  # TODO  
}
```

```
sub _on_mouse_move {  
  my ($self, $event) = @_;  
  # TODO  
}
```

```
sub _draw_crosshair_all {  
  my ($self) = @_;  
  # TODO  
}
```

```
sub set_timeframe {  
  my ($self, $tf) = @_;  
  # TODO  
}
```

```
sub reset_view {  
  my ($self) = @_;  
  # TODO  
}  
  
sub compute_intraday_labels {  
  my ($self) = @_;  
  # TODO  
}  
  
sub get_all_timestamps {  
  my ($self) = @_;  
  # TODO  
}  
  
1;
```

market.pl

Descripción

Punto de entrada del sistema. Controla el ciclo principal de ejecución y actualización de datos.

Tareas:

- Invoca la lectura de los datos (simulados o reales)
 - Invoca la actualización del mercado entre distintas temporalidades
 - Invoca la actualización de indicadores
 - Dibuja el primer chart
-